

— Módulo 4 · Procesamiento de Lenguaje Natural aplicado a Software

NLP Aplicado a Software — Semiteoría y Práctica

Ejercicio de Clase · Ejercicio 2 de 3 · Bloques 2.3 — 2.4 — 2.5 — 2.8 — Diseño y Aplicación

Nombre del alumno/a	Angel echenique
Fecha	26/04/2026
☐ Enlace GitHub del repositorio	

☐ **Instrucciones:** Ejercicio práctico conceptual. Diseña soluciones NLP reales, analiza código y razona tus decisiones técnicas. Cada bloque combina teoría aplicada con práctica guiada.

BLOQUE 1 — Análisis de Código NLP: Detecta y Corrige (2.3 + 2.8)

Lee el siguiente fragmento de código Python. Identifica TODOS los problemas que encuentras:

```
# archivo: sentiment_analyzer.py
from openai import OpenAI

client = OpenAI(api_key='sk-proj-abc123xyz...')

def analizar_sentimiento(texto):
    while True: # analiza continuamente
        response = client.chat.completions.create(
            model='gpt-4o',
            messages=[{'role': 'user', 'content': texto}],
            temperature=0.9
        )
        resultado = response.choices[0].message.content
        print(resultado)

# Ejecutar con todos los textos
textos = obtener_todos_los_textos_db() # puede ser 50,000 textos
for texto in textos:
    analizar_sentimiento(texto)
```

Nº	Problema detectado	¿Cómo lo corregirías?
1	La API key está escrita directamente en el código (<code>hardcoded</code>).	Guardarla en una variable de entorno como <code>OPENAI_API_KEY</code> y cargarla desde el entorno para no exponer credenciales.
2	La función tiene un <code>while True</code> que genera un bucle infinito.	Eliminar ese bucle y hacer que la función procese el texto una sola vez por llamada.
3	Usa <code>gpt-4o</code> para una tarea simple de análisis de sentimiento.	Cambiar a <code>gpt-4o-mini</code> , que es más rápido y más barato para clasificación sencilla.
4	No hay <code>system prompt</code> , así que el modelo no tiene reglas claras ni formato definido.	Añadir un <code>system prompt</code> que indique que debe analizar sentimiento y devolver solo JSON con campos concretos.
5	La <code>temperature=0.9</code> es demasiado alta para una tarea analítica.	Bajarla a <code>0.0</code> o <code>0.1</code> para obtener respuestas más consistentes y menos creativas.
6	No valida la respuesta de la API ni comprueba si el formato es correcto.	Verificar que la respuesta exista, que sea JSON válido y que contenga los campos esperados antes de usarla.
7	El resultado solo se imprime en consola y no se devuelve en un formato estructurado.	Hacer que la función devuelva un objeto o JSON con el resultado para poder validarlo, almacenarlo o usarlo en la aplicación.
8	No hay manejo de errores de API como <code>429</code> , <code>401</code> , <code>timeout</code> o errores <code>5xx</code> .	Añadir bloques <code>try/except</code> , reintentos controlados, registro de errores y respuestas alternativas según el tipo de fallo.
9	Procesa hasta 50.000 textos sin control de coste, rendimiento ni estrategia de procesamiento.	Aplicar procesamiento por lotes, limitar tokens, usar modelos más económicos y monitorizar consumo para evitar costes excesivos.

1.2 — Escribe las correcciones clave

Sin escribir el código completo, indica las partes/líneas críticas que cambiarías y por qué:

Corrección 1 — Seguridad de credenciales:

La clave de la API no debe estar escrita directamente en el archivo porque eso expone información sensible y es una mala práctica de seguridad. La cambiaría para cargarla desde una variable de entorno como `OPENAI_API_KEY`, de forma que el código sea más seguro y reutilizable.

Corrección 2 — Control del flujo de ejecución:

Eliminaría el `while True` dentro de la función, ya que provoca un bucle infinito y hace que cada texto se procese sin fin. La función debería analizar un texto una sola vez y devolver el resultado, dejando el control de iteración fuera de la función.

Corrección 3 — Mejor elección de modelo y parámetros:

Cambiaría `gpt-4o` por `gpt-4o-mini`, porque el análisis de sentimiento es una tarea simple de clasificación y no necesita un modelo más costoso. También bajaría la `temperature` a `0.0` o `0.1` para obtener respuestas más consistentes, predecibles y útiles en una aplicación.

Corrección 4 — Prompt claro y salida estructurada:

Añadiría un `system prompt` que defina el rol del modelo, la tarea exacta y el formato de salida esperado. Además, haría que la respuesta se devuelva en JSON con campos como `sentimiento`, `confianza` y `justificacion_breve`, para que la aplicación pueda validarla y usarla sin depender de texto libre.

Corrección 5 — Validación y manejo de errores:

Incorporaría validación de la respuesta y manejo de fallos de la API. Eso incluye comprobar si la salida es JSON válido, si contiene los campos esperados y manejar errores como `429`, `401`, `timeout` o `5xx` mediante `try/except`, reintentos controlados y registro de incidencias.

Corrección 6 — Escalabilidad y control de coste:

Como el sistema puede procesar `50.000` textos, implementaría una estrategia más eficiente: procesamiento por lotes, límites de tokens, uso de modelos económicos y monitorización del consumo. Así se evita saturar la API, se reducen costes y se mejora el rendimiento general del sistema.

BLOQUE 2 — Diseña el Flujo NLP Completo (2.3)

□ *Caso práctico: Una empresa de e-commerce quiere analizar automáticamente 500 reseñas diarias de productos. El sistema debe detectar sentimiento, extraer el producto mencionado, identificar el aspecto criticado (envío, calidad, precio) y generar una alerta si hay tendencia negativa.*

Completa las 7 fases del flujo NLP para este caso concreto:

Fase	Nombre de la fase	¿Qué hace en ESTE sistema?	Decisión técnica clave
1	Recepción del texto	El sistema recibe cada reseña escrita por el cliente desde la web, app o base de datos.	Centralizar las reseñas en un flujo de entrada uniforme antes de analizarlas.
2	Preprocesamiento	Limpia el texto, elimina ruido, espacios extra, caracteres innecesarios y recorta si es demasiado largo.	Normalizar el texto antes de enviarlo a la API para mejorar calidad y reducir coste.
3	Construcción del prompt	Se define un prompt con instrucciones claras para detectar sentimiento, producto, aspecto criticado y formato JSON.	Usar un <code>system prompt</code> preciso para asegurar respuestas estructuradas y consistentes.
4	Llamada a la API	Se envía cada reseña al modelo NLP con parámetros adecuados para clasificación y extracción.	Usar un modelo eficiente como <code>gpt-4o-mini</code> con <code>temperature</code> baja.
5	Validación respuesta	Se comprueba que la respuesta sea válida, completa y en JSON con los campos esperados.	Validar antes de usar la salida para evitar errores en el sistema.
6	Transformación	La respuesta se convierte en un formato interno que la app pueda guardar y analizar.	Mapear sentimiento, producto y aspecto a campos estructurados en base de datos.
7	Acción	El sistema guarda resultados, genera métricas y activa una alerta si detecta tendencia negativa repetida.	Definir umbrales de alerta, por ejemplo muchas reseñas negativas sobre el mismo producto o aspecto.

2.2 — Diseña el System Prompt

Escribe el system prompt completo para analizar reseñas. Recuerda: rol, formato JSON, campos y reglas:

```
Eres un sistema de análisis de reseñas para una tienda online.

Analiza el texto del usuario y devuelve ÚNICAMENTE un JSON con los siguientes campos:
- sentimiento: uno de ["positivo", "negativo", "neutral"]
- producto: nombre del producto mencionado o "desconocido" si no aparece claramente
- aspecto_criticado: uno de ["envio", "calidad", "precio", "atencion", "otro"]
- alerta: true o false
- razon_alerta: explicación breve de por qué se genera o no la alerta

Reglas:
- Si la reseña expresa satisfacción o valoración favorable, el sentimiento debe ser "positivo".
- Si expresa queja, frustración, fallo o decepción, el sentimiento debe ser "negativo".
- Si no hay una valoración clara, el sentimiento debe ser "neutral".
- Extrae el producto solo si aparece mencionado de forma explícita o claramente deducible.
- Identifica el principal aspecto criticado o valorado. Si no encaja en las categorías dadas, usa "otro".
- La alerta debe ser true solo si el sentimiento es negativo y hay un problema claro que debería revisarse.
- No añadas texto fuera del JSON.
- No inventes información que no esté en la reseña.
```

BLOQUE 3 — Diseña una Aplicación NLP Real (2.4 + 2.5)

□ *Caso práctico: Tu equipo quiere construir un sistema de análisis de contratos legales. Debe: (1) extraer cláusulas clave, fechas y partes, (2) detectar cláusulas de riesgo, (3) responder preguntas en lenguaje natural, (4) generar resumen ejecutivo de 5 puntos. Recibirá ~50 contratos/día en PDF.*

3.1 — Arquitectura: elige API y modelo para cada funcionalidad

Funcionalidad	API / Modelo elegido	Justificación técnica
Extracción de cláusulas y entidades (NER)	<code>gpt-4o-mini</code> o spaCy + OpenAI	Para extraer fechas, partes, montos y cláusulas relevantes hace falta precisión estructurada. Si se quiere flexibilidad sobre texto legal complejo, usaría OpenAI; si ya hay patrones repetitivos, spaCy puede complementar bien.
Detección de cláusulas de riesgo	<code>gpt-4o</code>	Esta tarea requiere comprender contexto jurídico, ambigüedad y matices del lenguaje legal, así que conviene un modelo más potente que uno básico.
Responder preguntas sobre el contrato	<code>gpt-4o</code> + embeddings	Para preguntas en lenguaje natural usaría RAG: primero recuperar fragmentos relevantes con embeddings y luego responder con un modelo conversacional.
Generar resumen ejecutivo	<code>gpt-4o</code>	Un resumen legal de calidad necesita buena comprensión del documento y capacidad de síntesis clara y fiable.
Análisis de imágenes (firmas, sellos)	<code>gpt-4o</code> multimodal	Si los contratos en PDF contienen imágenes, firmas o sellos, conviene un modelo multimodal que pueda interpretarlos junto al texto.

3.2 — Diseño modular (Principio de Responsabilidad Única)

Divide la aplicación en módulos. Para cada uno indica su responsabilidad única y qué llamada a la API realiza:

Nombre del módulo	Responsabilidad única (SRP)	Llamada a la API que realiza
<code>pdf_ingestor</code>	Recibir el contrato PDF y extraer su texto e imágenes.	No necesariamente llama al modelo; prepara la entrada para el resto del sistema.
<code>entity_extractor</code>	Extraer entidades clave como partes, fechas, montos y cláusulas importantes.	Llamada a <code>gpt-4o-mini</code> o pipeline de NER.
<code>risk_analyzer</code>	Detectar cláusulas problemáticas, penalizaciones, obligaciones o riesgos contractuales.	Llamada a <code>gpt-4o</code> con prompt especializado.
<code>semantic_retriever</code>	Buscar fragmentos relevantes del contrato para responder preguntas.	Llamada a modelo de embeddings como <code>text-embedding-3-small</code> .
<code>qa_assistant</code>	Responder preguntas del usuario usando el contexto recuperado del contrato.	Llamada a <code>gpt-4o</code> con contexto RAG.
<code>summary_generator</code>	Crear un resumen ejecutivo de 5 puntos.	Llamada a <code>gpt-4o</code> .

3.3 — Optimización de costes

Con 50 contratos/día, ¿cómo controlarías el coste? Describe 4 estrategias concretas:

Nº	Estrategia de optimización	¿Cómo se implementa concretamente?
1	Usar modelos pequeños para tareas simples	Utilizar <code>gpt-4o-mini</code> para extracción básica de entidades y reservar <code>gpt-4o</code> solo para tareas complejas como riesgo o QA legal.

2	Preprocesar y recortar el contenido	Limpiar el texto, eliminar páginas irrelevantes o repetidas y dividir contratos largos en fragmentos útiles antes de enviarlos a la API.
3	Usar embeddings para búsqueda previa	En vez de mandar el contrato completo en cada pregunta, primero recuperar solo los fragmentos relevantes con embeddings y luego enviarlos al modelo.
4	Evitar llamadas innecesarias y reutilizar resultados	Guardar embeddings, resúmenes y extracciones ya procesadas para no repetir análisis si el contrato vuelve a consultarse.

BLOQUE 4 — Manejo de Errores en NLP (2.8)

Las APIs de NLP pueden fallar por múltiples razones. Completa la tabla indicando causa, impacto y solución:

Error	¿Qué lo causa?	Impacto en la app	¿Cómo lo manejarías?
Rate limit (429)	Demasiadas peticiones en poco tiempo a la API.	La app no puede procesar solicitudes temporalmente y puede interrumpirse el flujo.	Aplicar reintentos con espera progresiva, controlar la velocidad de llamadas y procesar por lotes.
Timeout del servidor	La respuesta de la API tarda demasiado o el servidor no responde a tiempo.	El usuario puede quedar esperando o el proceso quedar incompleto.	Configurar timeout, reintentar un número limitado de veces y registrar el fallo para reanálisis posterior.
Error de autenticación (401)	API key incorrecta, caducada o mal configurada.	La aplicación no puede acceder al servicio NLP.	Verificar credenciales, usar variables de entorno y mostrar un error claro para revisión técnica.

Prompt demasiado largo (400)	Se envía más texto del permitido o demasiado contexto innecesario.	La petición falla y no se procesa el documento o consulta.	Recortar, resumir o fragmentar el texto antes de enviarlo al modelo.
Error del servidor (5xx)	Fallo interno temporal del proveedor de la API.	La app no obtiene respuesta aunque la petición esté bien hecha.	Reintentar con backoff, registrar el error y ofrecer una respuesta temporal o reintento posterior.
JSON inválido en respuesta	El modelo devuelve texto libre o una estructura mal formada.	La app no puede parsear ni usar correctamente la respuesta.	Validar la salida, intentar reprocesar y exigir en el prompt que responda solo en JSON.
Alucinación (datos inventados)	El modelo genera información no presente en el texto o interpreta demasiado.	La app puede guardar o mostrar datos falsos, generando errores de negocio.	Validar contra el texto original, limitar creatividad y usar prompts que prohíban inventar datos.

4.2 — Verdadero o Falso: Buenas Prácticas NLP

V / F	Afirmación	Justificación si es Falsa
F	Para clasificación simple, siempre es mejor usar GPT-4o que GPT-4o-mini.	Es falsa porque para tareas simples como clasificación o análisis básico suele ser mejor <code>gpt-4o-mini</code> , ya que es más barato, rápido y suficiente.
V	Pedir respuestas en JSON facilita la validación y el procesamiento en la app.	

F	Una temperature de 0.9 es la mejor opción para análisis de contratos legales.	Es falsa porque en tareas analíticas y legales conviene una temperature baja para obtener respuestas más precisas y consistentes.
V	Los embeddings son más eficientes que chat completions para búsquedas semánticas.	
F	Si la API no devuelve JSON válido, la aplicación debe fallar silenciosamente.	Es falsa porque debe detectarlo, registrar el error y manejarlo de forma controlada para no romper el sistema ni ocultar fallos.
F	Validar la salida de la IA antes de usarla es opcional si confías en el proveedor.	Es falsa porque siempre hay que validar la salida, incluso con proveedores fiables, para evitar errores de formato, datos incoherentes o alucinaciones.

BLOQUE 5 — Proyecto Integrador Final

□ *Caso práctico: Eres el desarrollador/a de una startup EdTech que quiere lanzar un MVP en 3 semanas de un asistente de IA para estudiantes. Debe: resumir apuntes en PDF, responder preguntas sobre el material, detectar lagunas de conocimiento y sugerir ejercicios personalizados. Presupuesto limitado, 200 usuarios/día en fase inicial.*

Decisión técnica	Tu respuesta justificada
¿Qué proveedor y modelo elegirías para cada funcionalidad?	Usaría OpenAI como proveedor principal para reducir complejidad de integración en el MVP. Para resumir apuntes en PDF y responder preguntas sobre el material elegiría <code>gpt-4o-mini</code> como opción base por coste y velocidad, y reservaría <code>gpt-4o</code> solo para casos donde el material sea especialmente complejo o la respuesta requiera más precisión. Para detectar lagunas de conocimiento combinaría extracción estructurada con <code>gpt-4o-mini</code> y búsqueda semántica con embeddings. Para sugerir ejercicios personalizados usaría <code>gpt-4o-mini</code> , porque la tarea requiere personalización pero no necesariamente el modelo más caro.

¿Qué parámetros clave configurarías (<code>temperature</code>, <code>max_tokens</code>, <code>system prompt</code>)?	Configuraría <code>temperature</code> baja en tareas analíticas: entre <code>0.0</code> y <code>0.2</code> para resumen, detección de lagunas y QA sobre contenido académico. Para sugerencias de ejercicios permitiría algo más de flexibilidad, por ejemplo <code>0.3</code> o <code>0.4</code>, sin perder control. <code>max_tokens</code> lo limitaría según tarea: resúmenes cortos entre <code>300</code> y <code>500</code>, respuestas a preguntas entre <code>200</code> y <code>400</code>, y sugerencias de ejercicios en un rango similar. El <code>system prompt</code> debe ser estricto: definir al asistente como tutor académico, exigir respuestas claras, no inventar contenido no presente en el material, y pedir formato estructurado cuando convenga, por ejemplo JSON para resultados internos.
¿Cómo estructurarías los módulos de la aplicación?	Separaría la aplicación en módulos con responsabilidad única. Un módulo de ingesta para recibir PDFs y extraer texto; un módulo de preprocesamiento para limpiar, fragmentar y normalizar contenido; un módulo de embeddings e indexación para almacenar fragmentos vectorizados; un módulo de resumen para generar síntesis del material; un módulo de preguntas y respuestas con enfoque RAG; un módulo de diagnóstico de lagunas para detectar temas no comprendidos o preguntas falladas; y un módulo de recomendación para sugerir ejercicios personalizados. Esta estructura facilita mantenimiento, pruebas y evolución del MVP sin rehacer todo el sistema.
¿Qué técnica NLP usarías para detectar lagunas de conocimiento?	Usaría una combinación de clasificación semántica y RAG. Haría que el sistema compare las preguntas del estudiante, sus respuestas o sus dudas con los conceptos clave del material usando embeddings. Así se pueden detectar temas que el alumno evita, responde mal o confunde. Después, usaría un modelo como <code>gpt-4o-mini</code> para transformar esa señal en un diagnóstico comprensible, por ejemplo: "domina definiciones básicas, pero tiene dificultades con embeddings y RAG". Esta técnica es mejor que una simple búsqueda por palabras porque detecta comprensión parcial aunque el alumno use otras expresiones.
¿Qué errores anticipas y cómo los manejarías?	Anticiparía cinco riesgos principales. Primero, PDFs mal extraídos o con texto incompleto: lo manejaría validando

	extracción y marcando documentos problemáticos para reproceso. Segundo, respuestas inventadas o no sustentadas: lo reduciría usando RAG, prompts estrictos y validación contra el material fuente. Tercero, JSON inválido en salidas internas: lo controlaría con validación y reintento automático. Cuarto, rate limits o timeouts: usaría colas, reintentos con backoff y procesamiento asíncrono. Quinto, costes crecientes por uso intensivo: monitorizaría tokens por usuario, limitaría longitud de contexto y reutilizaría embeddings y resúmenes ya generados.
¿Cómo controlarías el coste con 200 usuarios/día?	Controlaría coste con una estrategia de varias capas. Primero, usar <code>gpt-4o-mini</code> como modelo por defecto y escalar solo algunos casos a <code>gpt-4o</code>. Segundo, generar embeddings una sola vez por documento y reutilizarlos para todas las preguntas posteriores. Tercero, resumir y fragmentar el contenido para no enviar PDFs completos en cada interacción. Cuarto, cachear respuestas frecuentes y resultados intermedios como resúmenes o diagnósticos parciales. Quinto, limitar longitud de salida y número de consultas simultáneas por usuario en el MVP. Con 200 usuarios diarios, esta estrategia mantiene una experiencia buena sin disparar el presupuesto.